

LangChain 실습 퀴즈(문제)

Part 1: 기초 개념 및 설정

Q1. 다음 중 LangChain 설치 시 필요한 라이브러리가 아닌 것은?

- A) langchain
- B) openai
- C) python-dotenv
- D) tensorflow

답

D

**Q2. LangChain에서 환경 변수를 로드하기 위한 코드로 올바른 것은?

```
import os
from dotenv from load_dotenv

load_dotenv()

os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY")
```

**Q3. ChatOpenAI 모델을 초기화하는 올바른 방법은?

```
# OpenAI API 키를 설정한 후 ChatOpenAI 통해 객체 생성
from langchain.chat_models import ChatOpenAI

# LLM 모델 초기화
llm = ChatOpenAI(model_name='gpt-3.5-turbo')
```

Part 2: 프롬프트 템플릿

Q4. 다음 코드의 빈칸을 채워 프롬프트 템플릿을 완성하세요.

```
from langchain.prompts import ChatPromptTemplate

# 프롬프트 템플릿 정의
template = ""
```

당신은 {language} 전문가입니다.
{language}의 주요 특징과 장점에 대해 {num_points}가지 설명해주세요.
각 포인트는 번호를 매겨서 설명해주세요.
.....

```
prompt = ChatPromptTemplate.from_template(_____)

# 변수 값 설정
formatted_prompt = prompt._____(
    language="자바스크립트",
    num_points=3
)
```

정답

빈칸 : template / format

Part 3: 체인(Chains)

Q5. 다음 중 LLMChain의 역할로 올바른 것은?

- A) 여러 LLM 모델을 병렬로 실행한다
- B) 프롬프트 템플릿과 LLM을 연결하여 사용한다
- C) 대화 기록을 저장한다
- D) 웹 검색 결과를 가져온다

답

B

Q6. 다음 코드의 빈칸을 채워 간단한 LLMChain을 구현하세요.

```
from langchain.chains import LLMChain
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

# 프롬프트 템플릿 정의
template = "당신은 {topic} 전문가입니다. {topic}에 대해 간단히 설명해주세요."
prompt = ChatPromptTemplate.from_template(template)

# LLM 초기화
llm = ChatOpenAI(model_name="gpt-3.5-turbo")

# 체인 생성 # 위에 두개를 연결
chain = _____(llm=llm, prompt=prompt)
```

```
# 체인 실행
result = chain.invoke({"topic": "블록체인"})
```

답

LLMChain

Part 4: 메모리(Memory)

Q7. ConversationBufferMemory의 주요 목적은 무엇인가요?

- A) LLM의 모델 가중치를 저장한다
- B) 대화 기록을 유지하여 문맥을 기억한다
- C) 웹 검색 결과를 캐싱한다
- D) 임베딩 벡터를 저장한다

답

B

Q8. 다음 코드의 빈칸을 채워 대화 기억 기능을 구현하세요.

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain_openai import ChatOpenAI

# LLM 초기화
llm = ChatOpenAI(model_name="gpt-3.5-turbo")

# 메모리 초기화
memory = _____()

# 대화 체인 생성
conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

# 첫 번째 대화
response1 = conversation.invoke({"input": "안녕하세요! 제 이름은 김철수입니다."})

# 두 번째 대화 (이전 대화 기억)
```

```
response2 = conversation.invoke({"input": "제 이름이 뭐였죠?"})
```

답

ConversationBufferMemory

Part 5: 에이전트(Agents)

Q9. 다음 코드는 어떤 기능을 구현한 것인가요?

```
def calculator(expression):  
    """수학 표현식을 계산합니다."""  
    try:  
        return eval(expression)  
    except:  
        return "계산할 수 없는 표현식입니다."  
  
tools = [  
    Tool(  
        name="Calculator",  
        func=calculator,  
        description="수학 계산이 필요할 때 사용하세요."  
    )  
]  
  
agent = initialize_agent(  
    tools=tools,  
    llm=llm,  
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,  
    verbose=True  
)
```

- A) 웹 검색 에이전트
- B) 계산기 도구를 사용하는 에이전트
- C) 대화 메모리 에이전트
- D) 문서 검색 에이전트

답

B

에이전트 : 툴을 구현하고 사용해 어떤 역할을 할 수 있는 것

Q10. 에이전트를 초기화할 때 사용되는 `AgentType.ZERO_SHOT_REACT_DESCRIPTION`의 의미는 무엇인가요?

- A) 에이전트가 예제 없이도 추론하고 행동할 수 있다
- B) 에이전트가 여러 모델을 동시에 사용한다
- C) 에이전트가 대화 기록을 저장하지 않는다
- D) 에이전트가 오직 하나의 도구만 사용할 수 있다

답
A

Part 6: 문서 로더와 임베딩

Q11. 다음 코드에서 빈칸을 채워 문서 로더와 분할기를 구현하세요.

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter

# 문서 로더 사용
loader = _____("sample.txt")
documents = loader.load()

# 문서 분할
text_splitter = CharacterTextSplitter(chunk_size=100, chunk_overlap=0)
texts = text_splitter._____(documents)
```

답

```
TextLoader / split_documents
+ chunk_size : 문단을 글자만큼 잘라냄
+ chunk_overlap : 0 인 경우 순서가 저장 안됨
```

Q12. 벡터 저장소(Vector Store)의 주요 역할은 무엇인가요?

- A) LLM 모델 가중치 저장
- B) 사용자 입력 저장
- C) 임베딩된 문서 저장 및 유사도 검색
- D) API 키 안전 저장

답
C

Part 7: RetrievalQA 체인

Q13. RetrievalQA 체인의 주요 목적은 무엇인가요?

- A) 웹에서 정보를 검색하는 것
- B) 문서 검색과 질의응답을 결합하는 것
- C) 대화 기록을 저장하는 것
- D) 여러 LLM을 연결하는 것

답 :

B

Q14. 다음 코드의 빈칸을 채워 RetrievalQA 체인을 구현하세요.

```
from langchain.chains import RetrievalQA
from langchain.vectorstores import Chroma
from langchain_openai import ChatOpenAI

# 임베딩 생성 및 벡터 저장소 생성
embeddings = OpenAIEmbeddings()
db = Chroma.from_documents(texts, embeddings)

# LLM 초기화
llm = ChatOpenAI(model_name="gpt-3.5-turbo")

# RetrievalQA 체인 생성
qa = RetrievalQA._____(
    llm=llm,
    chain_type="stuff",
    retriever=db.as_retriever()
)

# 질문하기
query = "LangChain을 사용하면 어떤 이점이 있나요?"
result = qa.invoke({"query": query})
```

답

from_chain_type

Part 8: 웹 검색 통합

****Q15. LangChain에서 웹 검색을 위해 사용되는 라이브러리는?**

- A) WebSearchAPI
- B) GoogleSearch
- C) SerpAPIWrapper
- D) BingSearchWrapper

답
C

Part 9: 커스텀 체인

Q16. 다음 코드에서 `SequentialChain` 의 역할은 무엇인가요?

```
overall_chain = SequentialChain(
    chains=[topic_chain, selection_chain, outline_chain],
    input_variables=["domain"],
    output_variables=["topics", "selected_topic", "outline"],
    verbose=True
)
```

- A) 여러 체인을 병렬로 실행하는 것
- B) 여러 체인을 순차적으로 실행하고 출력을 연결하는 것
- C) 체인의 실행 결과를 저장하는 것
- D) 체인의 실행 속도를 향상시키는 것

답
B

Q17. `LLMChain` 을 생성할 때 `output_key` 매개변수의 역할은 무엇인가요?

- A) 체인의 출력을 저장할 파일 이름을 지정한다
- B) 체인의 출력을 지정된 이름으로 다음 체인에 전달하기 위해 사용한다
- C) API 키를 저장하는 데 사용한다
- D) 체인의 출력 결과가 저장될 딕셔너리의 키 이름을 지정한다

답
D

Part 10: 실전 애플리케이션

Q18. 다음 코드는 어떤 종류의 애플리케이션을 구현한 것인가요?

```
def chatbot_application():
    # LLM 초기화
    llm = ChatOpenAI(model_name="gpt-3.5-turbo")

    # 메모리 초기화
    memory = ConversationBufferMemory()

    # 챗봇 프롬프트 템플릿
    template = """
    당신은 친절하고 유능한 AI 비서입니다. 사용자의 질문에 명확하고 정확하게 답변해주세요.

    이전 대화:
    {history}

    사용자: {input}
    AI 비서:
    """

    prompt = ChatPromptTemplate.from_template(template)

    # 챗봇 체인 생성
    chatbot = LLMChain(
        llm=llm,
        prompt=prompt,
        memory=memory,
        verbose=True
    )
```

- A) 웹 검색 엔진
- B) 문서 요약기
- C) 대화형 AI 챗봇
- D) 코드 생성기

답
C

Q19. 챗봇 애플리케이션에서 **ConversationBufferMemory** 를 사용하는 주된 이유는 무엇인가요?

- A) 응답 속도를 높이기 위해
- B) API 비용을 절감하기 위해
- C) 이전 대화 내용을 기억하여 문맥을 유지하기 위해
- D) 웹 검색 결과를 저장하기 위해

답
C

실전 코딩 문제

Q20. 다음 요구사항에 맞는 **LangChain** 코드를 작성하세요:

- 사용자로부터 입력받은 주제에 대한 블로그 포스트 개요를 생성하는 체인을 구현하세요
- 체인은 제목, 서론, 3개의 섹션, 결론을 포함해야 합니다

```
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(model_name="gpt-3.5-turbo")

template = """
당신은 전문 블로그 작가입니다.
'{topic}'에 대한 블로그 포스트 개요를 작성해주세요.

다음 형식으로 작성해주세요:
# 제목

## 서론
- 핵심 포인트

## 섹션 1
- 주요 내용

## 섹션 2
- 주요 내용

## 섹션 3
- 주요 내용

## 결론
- 요약 및 마무리
"""

prompt = ChatPromptTemplate.from_template(template)

chain = LLMChain(llm=llm, prompt=prompt)

# 체인 실행
result = chain.invoke({"topic": topic})
return result["text"]
```

```
topic = input("블로그 주제를 입력하세요: ")
outline = create_blog_outline(topic)
print(outline)
```

심화 문제

Q21. LangChain을 사용하여 PDF 문서를 로드하고 질의응답하는 시스템을 구현하는 코드의 주요 단계를 순서대로 나열하세요.

1. pdf 문서를 로더
2. 문서를 분할(chuncking(칭킹), 오버랩)
3. 임베딩
4. 벡터저장소 생성
5. LLM 초기화
6. ... 쪽 진행